

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

A. Responsive Web Design (RWD)

RWD is an approach where a website's layout, content, and images automatically adjust to fit any screen size mobile, tablet, or desktop using one codebase instead of separate sites per device.

Why it matters: better user experience (no zooming/scrolling), easier maintenance (one codebase), better SEO (Google favors mobile-friendly sites), and cost savings over building multiple versions.

Key CSS Techniques

- **Fluid grids:** relative units (% , fr, vw) instead of fixed px
- **Flexible images:** `img { max-width: 100%; height: auto; }`
- **Media queries:** apply different styles at different breakpoints
- **Viewport meta tag:** `<meta name="viewport" content="width=device-width, initial-scale=1">`
- **Relative typography:** rem/em units that scale with settings
- **Mobile-first approach:** style for small screens first, enhance upward with min-width queries

Flexbox vs Grid

- **Flexbox** — one-dimensional (row or column); good for navbars, button rows, wrapping cards
- **Grid** — two-dimensional (rows + columns); good for full page layouts like header/sidebar/content/footer

Example

css

```
.layout {  
  
  display: grid;  
  
  grid-template-columns: 1fr;  
  
  grid-template-areas: "header" "nav" "main" "aside" "footer";  
  
}
```

```
@media (min-width: 768px) {  
  
  .layout {  
  
    grid-template-columns: 1fr 3fr;  
  
    grid-template-areas:  
  
      "header header"
```

```

    "nav nav"

    "main aside"

    "footer footer";

}

}

@media (min-width: 1024px) {

.layout {

    grid-template-columns: 200px 1fr 250px;

    grid-template-areas:

        "nav header header"

        "nav main aside"

        "nav footer footer"; }}

```

Stacks vertically on mobile → sidebar appears on tablet → persistent nav column added on desktop.

UX Improvements

- Touch targets $\geq 44 \times 44$ px
- `clamp()` for smooth font scaling
- `srcset/<picture>` for responsive images
- Hamburger menu below ~ 768 px
- Lazy-load content for performance on mobile networks
- Test on real devices, not just DevTools resizing

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Client-Side Validation

Client-side validation checks user input in the browser (before it's sent to the server) to catch errors early — empty fields, wrong format, invalid characters. It improves user experience by giving instant feedback without a round trip to the server, but it should never replace server-side validation, since client-side code can be bypassed or disabled.

Purpose: faster feedback, reduced server load, better UX, and a first line of defense against bad/malicious input.

Role of JavaScript Event Handling

Validation relies on listening for events on form elements:

- submit — validate all fields before allowing form submission (and `preventDefault()` if invalid)
- input/change — validate a field as the user types or leaves it
- blur/focus — validate when a field loses/gains focus, useful for per-field error messages

Role of Regular Expressions

Regex checks if input matches an expected pattern — useful for structured data like emails, phone numbers, or postal codes where format matters more than just "not empty."

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
const phonePattern = /^\d{10}$/;
```

Example

```
<form id="myForm">
```

```
<input type="text" id="email" placeholder="Email">
```

```
<span id="emailError" style="color:red;"></span>
```

```
<input type="text" id="phone" placeholder="Phone">
```

```
<span id="phoneError" style="color:red;"></span>
```

```
<button type="submit">Submit</button>
```

```
</form>
```

```
const form = document.getElementById('myForm');
```

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
const phonePattern = /^d{10}$/;
```

```
form.addEventListener('submit', function (e) {
```

```
  e.preventDefault();
```

```
  let isValid = true;
```

```
  const email = document.getElementById('email').value;
```

```
  const emailError = document.getElementById('emailError');
```

```
  if (!emailPattern.test(email)) {
```

```
    emailError.textContent = 'Invalid email';
```

```
    isValid = false;
```

```
  } else {
```

```
    emailError.textContent = '';
```

```
}
```

```
const phone = document.getElementById('phone').value;
```

```
const phoneError = document.getElementById('phoneError');
```

```
if (!phonePattern.test(phone)) {
```

```
    phoneError.textContent = 'Phone must be 10 digits';
```

```
    isValid = false;
```

```
} else {
```

```
    phoneError.textContent = "";
```

```
}
```

```
if (isValid) form.submit();
```

```
});
```

Improvements

Usability:

- Validate on blur/input for real-time feedback, not just on submit
- Use HTML5 attributes (required, type="email", pattern) as a first layer
- Clear, specific error messages near each field, not just red borders

Security:

- Always re-validate on the server — client-side checks are bypassable
- Sanitize/escape input server-side to prevent injection attacks (SQL injection, XSS)
- Use HTTPS to protect data in transit

- Rate-limit submissions to prevent abuse/bots (e.g., CAPTCHA)

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

CSS Box Model

Every element is a rectangular box made of four layers, from inside out:

- Content — the actual text/image
- Padding — space between content and border
- Border — the edge around padding
- Margin — space outside the border, separating it from other elements

By default, width/height apply only to content, so padding and border add extra size — this is a common cause of layout breaking. Fix: box-sizing: border-box; makes width/height include padding and border.

CSS Positioning

- static — default, normal document flow
- relative — positioned relative to its own normal position
- absolute — removed from flow, positioned relative to nearest positioned ancestor
- fixed — positioned relative to the viewport, stays on scroll
- sticky — toggles between relative and fixed based on scroll position

Absolute/fixed elements can overlap or escape their container if not handled carefully, breaking layout on resize.

Causes of Layout Breaking

- Fixed px widths instead of relative units (% , fr, vw)
- Not using box-sizing: border-box
- Misused position: absolute/fixed without a positioned parent
- Missing viewport meta tag on mobile
- Content (long text/images) overflowing fixed-size containers
- No media queries for smaller breakpoints

Media Queries as a Fix

Media queries apply different CSS rules at different viewport widths, letting layout restructure (e.g., multi-column → single column) instead of squeezing or overflowing:

```
@media (max-width: 768px) {  
  
  .container { flex-direction: column; width: 100%; }  
  
}
```

Best Practices

- Use box-sizing: border-box globally
- Use relative units and fluid grids (Flexbox/Grid) over fixed px
- Add the viewport meta tag
- Use max-width instead of fixed width for containers/images
- Test across breakpoints with mobile-first media queries

Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web

development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Document Object Model (DOM)

The DOM is a tree-like representation of an HTML document that the browser builds from the page's markup, where each element, attribute, and piece of text is a node. It acts as an interface between the webpage and scripting languages like JavaScript, allowing content to be read and changed dynamically **without reloading the page**.

JavaScript DOM Manipulation

JavaScript can select, modify, add, or remove elements using DOM methods:

- **Selecting:** `document.getElementById()`, `querySelector()`, `querySelectorAll()`
- **Changing content:** `element.textContent`, `element.innerHTML`
- **Changing styles/attributes:** `element.style.color`, `element.setAttribute()`
- **Adding/removing elements:** `createElement()`, `appendChild()`, `removeChild()`
- **Responding to events:** `addEventListener()` to trigger updates on user actions (clicks, input, etc.)

Example

```
<button id="loadBtn">Load Message</button>
```

```
<p id="output"></p>
```

```
document.getElementById('loadBtn').addEventListener('click', function () {  
  
    const output = document.getElementById('output');  
  
    output.textContent = 'Content updated dynamically!';  
  
    output.style.color = 'green';  
  
});
```

Clicking the button updates the paragraph's text instantly, with no page reload.

Improvements

Performance:

- Minimize direct DOM manipulation in loops; batch updates using a DocumentFragment
- Use event delegation (one listener on a parent) instead of many individual listeners
- Cache DOM references in variables instead of repeated querySelector calls

User Experience:

- Show loading indicators for async content (e.g., fetched via fetch())
- Use smooth transitions/animations for content changes
- Ensure updates are accessible (e.g., aria-live regions for screen readers)

Code of Conduct:

I Avinash.K(192411044) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*Avinash K*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with	Incorrect or irrelevant answer

				noticeable errors	
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand